

**INFORME DEL
PROYECTO INTEGRADOR DE SISTEMA
DE IA APLICADA PARA ANÁLISIS
PREDICTIVO**

TÍTULO: Reconocimiento de números
escritos a mano mediante Inteligencia
artificial (CNN)

Elaborado por:

Gabriela Magdalena Andaluz Ortiz

Fecha: 30 de septiembre de 2026

Tabla de Contenido	¡Error! Marcador no definido.
1. RESUMEN	3
2. METODOLOGIA APLICADA.....	4
3. RESULTADOS Y VISUALIZACIÓN	8
4. REFLEXIONES ÉTICAS SOBRE USO DE LA IA	11
5. REPOSITORIO GitHub	11
6. CONCLUSIONES	11

1. TEMA: Reconocimiento de números escritos a mano mediante Inteligencia artificial (CNN)

2. RESUMEN

El presente proyecto desarrolla una aplicación de Inteligencia Artificial orientada al reconocimiento automático de números escritos a mano, mediante el uso de una Red Neuronal Convolutiva (Convolutional Neural Network, CNN). El objetivo principal consiste en entrenar un modelo capaz de recibir como entrada la imagen de un dígito manuscrito y clasificarlo correctamente dentro de una de las diez categorías correspondientes a los números del 0 al 9.

Para el desarrollo se utilizó el conjunto de datos MNIST (Modified National Institute of Standards and Technology), ampliamente empleado como referencia en problemas de clasificación de imágenes y aprendizaje profundo. Este conjunto contiene imágenes en escala de grises de dígitos escritos a mano, representadas mediante matrices de 28×28 píxeles. La implementación se realizó en Python, utilizando las librerías TensorFlow/Keras, NumPy y Matplotlib, y se ejecutó mediante Google Colab. El conjunto MNIST proporciona 60000 imágenes para entrenamiento y 10000 imágenes para pruebas. Antes del entrenamiento, los valores de intensidad de los píxeles fueron normalizados al intervalo $[0,1]$ y las etiquetas de las clases fueron transformadas mediante codificación categórica.

La arquitectura implementada utiliza dos capas convolucionales para la extracción automática de características de las imágenes, capas de reducción dimensional mediante Max Pooling y un conjunto de capas completamente conectadas para efectuar la clasificación final. La salida está constituida por diez neuronas con activación Softmax, cada una asociada a uno de los posibles dígitos.

El modelo fue entrenado durante 20 épocas utilizando el optimizador Adam y la función de pérdida categorical cross-entropy. Finalmente, su capacidad de generalización fue evaluada sobre las 10000 imágenes del conjunto de prueba. Adicionalmente, se realizaron predicciones individuales sobre diferentes imágenes para comparar visualmente la clase estimada por la red neuronal con el valor real. Los resultados obtenidos permiten comprobar la aplicabilidad de las redes neuronales convolucionales para resolver problemas de reconocimiento de patrones visuales y constituyen un ejemplo práctico del uso de técnicas de Inteligencia Artificial y Deep Learning para automatizar tareas de clasificación.

3. METODOLOGIA APLICADA

2.1. Descripción general

Para resolver el problema de reconocimiento de números manuscritos se utilizó una metodología de aprendizaje supervisado, debido a que cada imagen disponible durante el entrenamiento se encuentra asociada con una etiqueta que identifica el número representado.

El procedimiento seguido puede resumirse en las siguientes etapas:

1. Carga del conjunto de datos MNIST.
2. Separación de los datos de entrenamiento y prueba.
3. Preprocesamiento y normalización de las imágenes.
4. Codificación de las clases.
5. Visualización preliminar de los datos.
6. Diseño de la Red Neuronal Convolucional.
7. Configuración y entrenamiento del modelo.
8. Evaluación utilizando datos no empleados directamente para el entrenamiento.
9. Predicción de números manuscritos.
10. Visualización y análisis de los resultados.

2.2. Base de datos MNIST

Para el desarrollo de la aplicación se utilizó la base de datos MNIST, disponible directamente mediante Keras: `keras.datasets.mnist.load_data()`. La base de datos contiene imágenes correspondientes a diez clases: **0, 1, 2, 3, 4, 5, 6, 7, 8 y 9**. Cada imagen tiene una resolución de 28×28 píxeles y se encuentra representada en escala de grises.

El conjunto utilizado se encuentra distribuido de la siguiente manera:

Conjunto	Número de imágenes	Utilización
Entrenamiento	60000	Aprendizaje de los parámetros del modelo
Prueba	10000	Evaluación final del modelo
Total	70000	Base de datos completa

Adicionalmente, durante el entrenamiento se utiliza: `validation_split=0.1`. Por lo tanto, Keras reserva el 10 % del conjunto proporcionado a `model.fit()` para validación. De las 60000 muestras inicialmente destinadas al entrenamiento,

aproximadamente 54000 se utilizan para ajustar los parámetros del modelo y 6000 para validación. Esta separación permite observar el comportamiento del modelo sobre información diferente de aquella utilizada directamente para actualizar sus parámetros durante cada época.

2.3. Preprocesamiento de las imágenes

Originalmente, los píxeles de las imágenes presentan valores de intensidad comprendidos entre 0 y 255. Para facilitar el proceso de aprendizaje, estos valores fueron normalizados al intervalo [0,1] mediante:

```
x_train = x_train.astype("float32") / 255
x_test = x_test.astype("float32") / 255
```

La normalización evita trabajar con valores de entrada innecesariamente elevados y contribuye a mejorar el comportamiento numérico durante el entrenamiento. Posteriormente, se añadió una dimensión adicional a cada imagen:

```
np.expand_dims(x_train, -1)
```

De esta manera, cada muestra presenta una dimensión: $28 \times 28 \times 1$. Donde 28×28 representa la resolución espacial y 1 corresponde al único canal de la imagen en escala de grises.

2.4. Codificación de las clases

Las etiquetas originales corresponden a valores enteros comprendidos entre 0 y 9. Para utilizar la función de pérdida categórica seleccionada, estas etiquetas se convierten a una representación one-hot encoding mediante:

```
keras.utils.to_categorical(y_train, num_classes)
```

Por ejemplo, una muestra perteneciente al número 3 se representa como: [0, 0, 0, 1, 0, 0, 0, 0, 0, 0]. Esto permite relacionar directamente las diez posibles categorías con las diez neuronas de la capa de salida.

2.5. Visualización de las muestras

Antes de entrenar el modelo se visualizan nueve imágenes del conjunto de entrenamiento en una matriz de 3×3 .

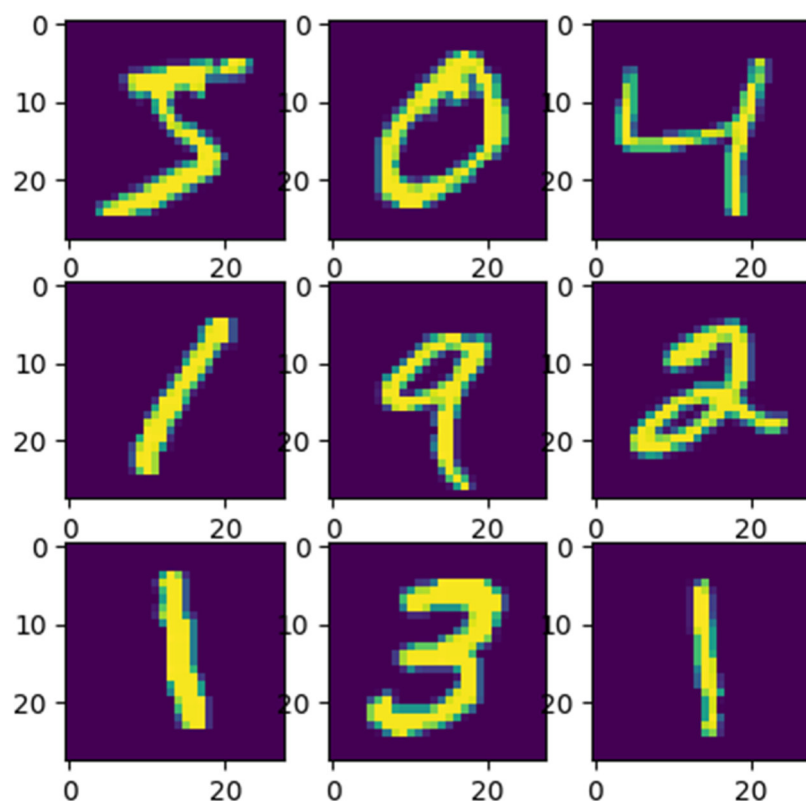


Figura 1. Ejemplos de números manuscritos pertenecientes al conjunto de datos MNIST.

Esta visualización permite verificar que las imágenes contienen diferentes estilos de escritura, formas y características para cada número. Estas variaciones hacen que la clasificación no pueda depender exclusivamente de una comparación exacta de píxeles y justifican el empleo de técnicas de aprendizaje profundo capaces de extraer características relevantes.

2.6. Arquitectura de la Red Neuronal Convolutiva

Para realizar el reconocimiento se implementó una CNN, arquitectura especialmente apropiada para procesamiento y clasificación de imágenes. La estructura general utilizada es:

Imagen $28 \times 28 \times 1 \rightarrow \text{Conv2D} \rightarrow \text{MaxPooling} \rightarrow \text{Conv2D} \rightarrow \text{MaxPooling} \rightarrow \text{Flatten} \rightarrow \text{Dense} \rightarrow \text{Dense} \rightarrow \text{Dense} \rightarrow \text{Softmax}$

La arquitectura implementada puede detallarse de la siguiente manera:

Etapa	Capa	Características principales
Entrada	Input	$28 \times 28 \times 1$
1	Conv2D	6 filtros de 5×5 , ReLU

2	MaxPooling2D	Ventana 2×2
3	Conv2D	16 filtros de 5×5 , ReLU
4	MaxPooling2D	Ventana 2×2
5	Flatten	Conversión a vector
6	Dense	256 neuronas
7	Dense	100 neuronas
8	Dense	100 neuronas
9	Dense	10 neuronas, Softmax

Las primeras capas convolucionales son responsables de extraer automáticamente características visuales de las imágenes. Durante el aprendizaje, sus filtros pueden responder a patrones útiles para distinguir los dígitos, como bordes, curvas, trazos y combinaciones espaciales.

La función de activación ReLU (Rectified Linear Unit) se utiliza en las capas convolucionales para introducir no linealidad en el modelo.

Las capas MaxPooling2D reducen las dimensiones espaciales de las representaciones intermedias. Esto disminuye la cantidad de información que debe procesarse en las siguientes etapas y conserva las características predominantes detectadas por las convoluciones.

Después de las etapas convolucionales, Flatten convierte las características obtenidas en un vector que puede ser procesado por las capas completamente conectadas.

Finalmente, la última capa dispone de 10 neuronas, una para cada posible número, y utiliza la función de activación Softmax. Esta función transforma la salida de la red en una distribución de probabilidades entre las diez clases.

Por tanto, la clase con mayor probabilidad es seleccionada como la predicción realizada por el sistema:

$$\text{Número predicho} = \arg \max (\text{salida Softmax})$$

2.7. Configuración y entrenamiento

Para entrenar la red se utilizaron los siguientes parámetros:

Parámetro	Valor
Número de clases	10
Tamaño de entrada	28x28x1
Batch size	100

Número de épocas	20
Optimizador	Adam
Función de pérdida	Categorical Cross-Entropy
Métrica principal	Accuracy
Validación	10 % del conjunto de entrenamiento

El modelo se compila mediante:

```
model.compile(loss="categorical_crossentropy", optimizer="adam",
              metrics=["accuracy"])
```

El entrenamiento se realiza durante 20 épocas, procesando las imágenes en lotes de 100 muestras. El optimizador Adam actualiza iterativamente los parámetros de la red con el propósito de minimizar la función de pérdida. En este caso se utiliza `categorical_crossentropy`, apropiada para un problema de clasificación multiclase con las etiquetas codificadas mediante one-hot encoding.

4. RESULTADOS Y VISUALIZACIÓN

3.1. Evaluación del modelo

Una vez finalizado el entrenamiento, el desempeño de la red se evalúa utilizando las 10 000 imágenes del conjunto de prueba, las cuales no son utilizadas para actualizar los parámetros del modelo durante el entrenamiento. La evaluación se realiza mediante:

```
model.evaluate(x_test, y_test, verbose=0)
```

Los principales indicadores obtenidos son:

Indicador	Resultado
Test Loss	0.04664
Test Accuracy	0.9862
Test Accuracy (%)	98.62%

La métrica `accuracy` representa la proporción de imágenes clasificadas correctamente. Colab reporta un valor de 0.9862, esto significaría que aproximadamente el 98.62 % de las imágenes del conjunto de prueba fueron clasificadas correctamente.

La función loss, por su parte, cuantifica el error asociado con las probabilidades generadas por la red. Su interpretación debe realizarse conjuntamente con el accuracy y, de ser posible, con el comportamiento observado durante entrenamiento y validación.

3.2. Predicción de imágenes individuales

Para comprobar visualmente el funcionamiento del sistema, el programa selecciona las primeras nueve imágenes del conjunto de prueba y calcula:

```
resultados = model.predict(x_test[0:9])
```

Para cada imagen se determina la clase con mayor probabilidad mediante:

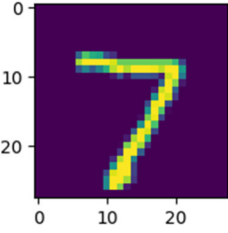
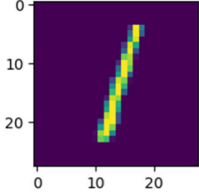
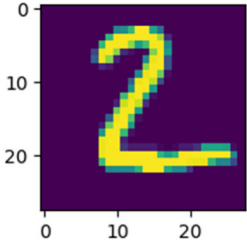
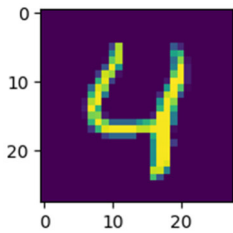
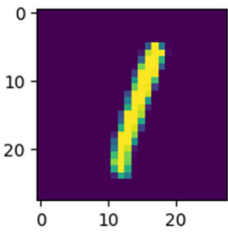
```
resultados[i].argmax()
```

Posteriormente, el resultado se compara con la etiqueta real:

```
y_test[i].argmax()
```

El procedimiento permite presentar para cada muestra:

Predicción del modelo → Valor real → Imagen manuscrita

Predicción: 7 Real: 7 	Predicción: 1 Real: 1 	Predicción: 2 Real: 2 
Predicción: 0 Real: 0 	Predicción: 4 Real: 4 	Predicción: 1 Real: 1 

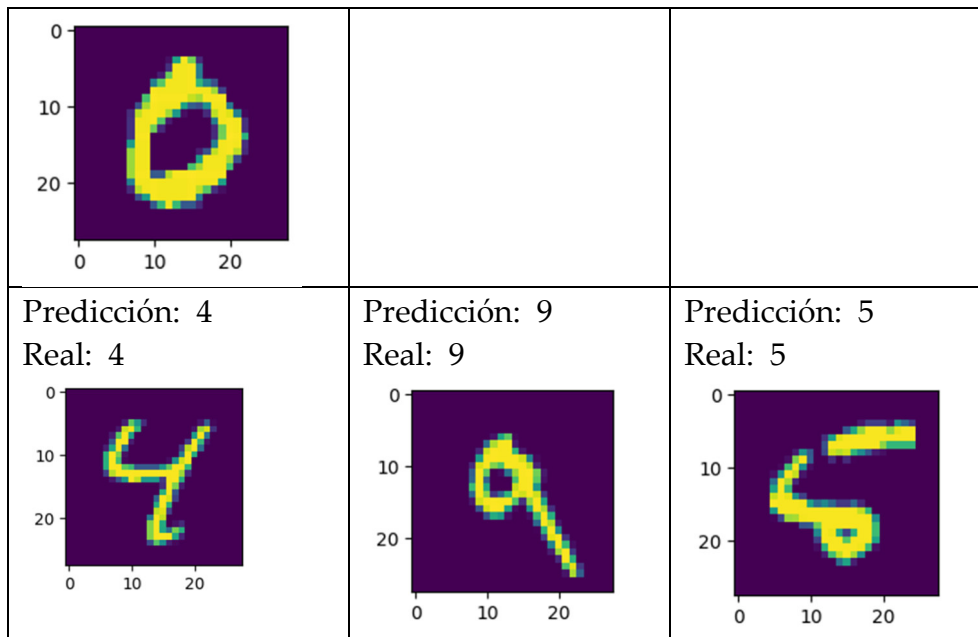


Figura 2. Ejemplos de clasificación de números manuscritos del conjunto de prueba.

Una coincidencia entre el número predicho y el número real representa una clasificación correcta. Las diferencias entre ambos valores permiten identificar los casos en los cuales el modelo presenta dificultades.

3.3. Análisis de los resultados

La aplicación implementada demuestra el funcionamiento de una red neuronal convolucional en un problema real de reconocimiento de imágenes. La CNN aprende directamente a partir de los ejemplos disponibles, evitando la necesidad de definir manualmente características geométricas para cada uno de los números.

Una característica relevante es que el sistema no memoriza únicamente una representación específica de cada número. Durante el entrenamiento recibe miles de ejemplos con diferentes características de escritura, lo que permite aprender patrones comunes que posteriormente son utilizados para clasificar imágenes no empleadas directamente para el ajuste de los parámetros.

La evaluación con el conjunto de prueba es particularmente importante porque permite estimar la capacidad de generalización de la red neuronal. Un buen desempeño sobre los datos de entrenamiento no sería suficiente si el modelo no pudiera reconocer correctamente nuevas imágenes.

5. REFLEXIONES ÉTICAS SOBRE USO DE LA IA

El uso de Inteligencia Artificial para el reconocimiento de números manuscritos ofrece ventajas importantes en la automatización de tareas; sin embargo, su desempeño depende de la calidad y representatividad de los datos de entrenamiento. Aunque MNIST contiene una amplia variedad de ejemplos, no representa necesariamente todos los estilos de escritura ni las condiciones que pueden encontrarse en una aplicación real. Por ello, una alta precisión obtenida con esta base de datos no garantiza el mismo desempeño con imágenes externas.

También debe considerarse que una red neuronal puede cometer errores de clasificación, especialmente ante imágenes ambiguas, incompletas o diferentes de las utilizadas durante el entrenamiento. Por esta razón, es importante conocer las limitaciones del modelo y mantener supervisión humana cuando este tipo de tecnología se utilice en aplicaciones donde una clasificación incorrecta pueda tener consecuencias relevantes.

Finalmente, el desarrollo responsable de sistemas de IA requiere considerar aspectos de transparencia, privacidad y responsabilidad. Se debe documentar el origen de los datos, el funcionamiento general del modelo y sus limitaciones, además de proteger cualquier información personal que pudiera procesarse. La IA debe utilizarse como una herramienta de apoyo, considerando tanto su desempeño técnico como las implicaciones éticas de su aplicación.

6. REPOSITORIO GitHub

El archivo principal “Reconocimiento_Numeros_MNIST.ipynb” contendrá el código desarrollado en Google Colab.

Enlace al repositorio

GitHub: https://github.com/Gaby-ia/Proyecto_COLAB_UIDE

7. CONCLUSIONES

En este proyecto se desarrolló un sistema de Inteligencia Artificial para el reconocimiento de números escritos a mano utilizando una Red Neuronal Convolutiva y el conjunto de datos MNIST. La implementación permitió integrar las principales etapas de un problema de aprendizaje supervisado:

preparación de datos, diseño del modelo, entrenamiento, validación, evaluación y realización de predicciones.

La utilización de capas convolucionales permite extraer automáticamente características relevantes de las imágenes, mientras que las capas completamente conectadas realizan la clasificación final entre las diez categorías correspondientes a los números del 0 al 9.

La evaluación sobre un conjunto independiente de 10000 imágenes permite analizar la capacidad de generalización del modelo. 98.62% obtenido experimentalmente evidencia el desempeño alcanzado por la arquitectura implementada.

El proyecto demuestra, además, que el desarrollo de aplicaciones de Inteligencia Artificial debe considerar aspectos que van más allá del desempeño numérico. La calidad de los datos, los posibles sesgos, la privacidad, la transparencia, la supervisión humana y las consecuencias de los errores constituyen elementos fundamentales para garantizar un uso responsable de estas tecnologías.